Binary Search Tree

C Program Implementation and Execution Output

PROGRAM SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a BST node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Function to insert a value into the BST
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    } else {
        printf("Duplicate value %d ignored.\n", value);
    }
    return root;
}

// Function to search for a value in the BST
struct Node* search(struct Node* root, int key) {
    if (root == NULL || root->data == key) {
        return root;
    }
    if (key < root->data) {
        return search(root->left, key);
    }
    return search(root->right, key);
}

// Inorder traversal (Left, Root, Right)
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Preorder traversal (Root, Left, Right)
```

PROGRAM OUTPUT

```
Inorder Traversal: 10 30 50 70 80
Preorder Traversal: 50 10 30 70 80
Postorder Traversal: 30 10 80 70 50

Enter value to search: 30
Element 30 found in the BST.

Inorder Traversal: 10 30 50 70 80
Preorder Traversal: 50 10 30 70 80
Postorder Traversal: 30 10 80 70 50

Enter value to search: 55
Element 55 NOT found in the BST.
```